

Documentación de ByteKnight

Objetivo: Despliegue de una infraestructura de red segura en AWS (VPC) que aloja un servidor web Apache con almacenamiento externo (EBS).

Integrantes:

1. Benjamin Uribe
2. Max Coñoman

A. Diagrama de Infraestructura

B. Explicación

Antes de explicar cómo funciona cada función, decidimos utilizar una instancia t3.micro por su relación precio-rendimiento.

También se implementó un script [destroy.sh](#) para limpiar cada instancia una vez sea utilizada, con esto nos aseguramos de que no queden rastros de la instancia, VPC y los gateways, para que no queden recursos activos.

Para referirnos a Ethical hacking, se creó una corporación que busca ayudar y entregar servicios de ciberseguridad para la protección de datos de tanto pequeñas como grandes empresas.

Esta corporación nació por la convicción de que las mejores defensas se construyen pensando como el atacante. Para así poder entregar la mayor seguridad de nuestros clientes.

Configuraciones

Lo primero que hacemos es configurar la región.

```
REGION="us-east-1"
AZ1="us-east-1a"
AZ2="us-east-1b"
aws configure set default.region $REGION
```

VPC

Se crea la VPC tomando de CIDR 10.0.0.0/16

```
echo "Creando VPC..."
export VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
| --query 'Vpc.VpcId' --output text)
```

SubRedes

Se ha diseñado una red aislada para garantizar la seguridad de los recursos.

- Región de despliegue: **us-east-1** (N. Virginia).
- Segmento de Red (VPC): CIDR **10.0.0.0/16**.
- Subredes:
 - Pública: Ubicada en **us-east-1a** (**10.0.1.0/24**) para el servidor web con acceso a internet.
 - Privada: Ubicada en **us-east-1b** (**10.0.2.0/24**) para asegurar datos sensibles.

Creamos una SubRed Pública

```
echo "Creando subred pública..."
export SUBNET_PUB=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.1.0/24 --availability-zone $AZ1 \
  --query 'Subnet.SubnetId' --output text)

aws ec2 create-tags --resources $SUBNET_PUB --tags Key=Name,Value=ethicalHackin-subred-publica
aws ec2 modify-subnet-attribute --subnet-id $SUBNET_PUB --map-public-ip-on-launch

echo "Subred Pública creada: $SUBNET_PUB"
```

Creamos una SubRed Privada

```
echo "Creando subred privada..."
export SUBNET_PRV=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.2.0/24 --availability-zone $AZ2 \
  --query 'Subnet.SubnetId' --output text)

aws ec2 create-tags --resources $SUBNET_PRV --tags Key=Name,Value=ethicalHackin-subred-privada

echo "Subred Privada creada: $SUBNET_PRV"
```

Tabla de ruteo

La seguridad se gestiona mediante una Tabla de Ruteo y un Grupo de Seguridad (Security Group).

Según las reglas de entrada (Ingress) aplicadas:

Se aplicó el principio de "mínimo privilegio":

- Puerto 80 (HTTP): Abierto al mundo (**0.0.0.0/0**) para permitir el tráfico web.
- Puerto 22 (SSH): Restringido exclusivamente a la IP pública del administrador para mantenimiento seguro.

```

echo "Creando tabla de ruteo..."
export RT_ID=$(aws ec2 create-route-table --vpc-id $VPC_ID \
| --query 'RouteTable.RouteTableId' --output text)

aws ec2 create-tags --resources $RT_ID --tags Key=Name,Value=ethicalHackin-rt

aws ec2 create-route --route-table-id $RT_ID \
| --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW_ID

aws ec2 associate-route-table --route-table-id $RT_ID --subnet-id $SUBNET_PUB

echo "Tabla de Ruteo configurada: $RT_ID"

```

Security group

```

echo "Creando Security Group..."
Corregido: se eliminaron espacios en --group-name
export SG_ID=$(aws ec2 create-security-group --group-name "ethicalHackin-sg" \
| --description "Security Group para aplicacion ethicalHackin" \
| --vpc-id $VPC_ID --query 'GroupId' --output text)

aws ec2 create-tags --resources $SG_ID --tags Key=Name,Value=ethicalHackin-sg

aws ec2 authorize-security-group-ingress --group-id $SG_ID \
| --protocol tcp --port 80 --cidr 0.0.0.0/0

IP_PUBLICA=$(curl -s -4 ifconfig.me)
aws ec2 authorize-security-group-ingress --group-id $SG_ID \
| --protocol tcp --port 22 --cidr ${IP_PUBLICA}/32

echo "Security Group configurado: $SG_ID"

```

Lanzar instancia EC2

Se despliega una instancia **t3.micro** utilizando una imagen Amazon Linux.

Configuración Automatizada (User Data)

Al iniciar, la instancia ejecuta automáticamente las siguientes tareas:

1. Actualización de sistema: **yum update**.
2. Servidor Web: Instalación, inicio y habilitación de **httpd** (Apache).
3. Gestión de Almacenamiento: Identifica el disco adicional, lo formatea en **ext4** y lo monta en **/mnt/datos**.
4. Despliegue de Contenido: Crea un enlace simbólico para que el contenido web resida en el volumen persistente.

```

echo "Lanzando instancia EC2..."
export INSTANCE_ID=$(aws ec2 run-instances \
  --image-id ami-0a29987e14ae814db \
  --instance-type t3.micro \
  --key-name nuevos \
  --subnet-id $SUBNET_PUB \
  --security-group-ids $SG_ID \
  --associate-public-ip-address \
  --user-data '#!/bin/bash'
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
while [ ! -b /dev/nvme1n1 ] && [ ! -b /dev/xvdf ]; do sleep 5; done
if [ -b /dev/nvme1n1 ]; then DISCO="/dev/nvme1n1"; else DISCO="/dev/xvdf"; fi
mkfs -t ext4 $DISCO
mkdir -p /mnt/datos
mount $DISCO /mnt/datos
echo "<h1>Bienvenido al sistema ethicalHackin</h1>" > /mnt/datos/index.html
rm -rf /var/www/html
ln -s /mnt/datos /var/www/html' \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 create-tags --resources $INSTANCE_ID --tags Key=Name,Value=ethicalHackin-webserver

echo "Lanzando instancia EC2: $INSTANCE_ID..."

```

Esperamos a que la instancia esté lista

```

echo "Esperando a que la instancia se inicialice..."
aws ec2 wait instance-running --instance-ids $INSTANCE_ID

```

Creamos y adjuntamos el volumen EBS

Para evitar la pérdida de datos al apagar la instancia, se utiliza un volumen EBS de 8GB tipo **gp3**. Este volumen se adjunta dinámicamente como **/dev/sdf**.

```

echo "Creando volumen EBS de 8GB..."
export VOL_ID=$(aws ec2 create-volume --size 8 --volume-type gp3 \
  --availability-zone $AZ1 --query 'VolumeId' --output text)

aws ec2 create-tags --resources $VOL_ID --tags Key=Name,Value=ethicalHackin-datos

echo "Esperando a que el volumen esté disponible..."
aws ec2 wait volume-available --volume-ids $VOL_ID

echo "Adjuntando volumen..."
aws ec2 attach-volume --volume-id $VOL_ID --instance-id $INSTANCE_ID --device /dev/sdf

echo "Volumen $VOL_ID adjuntado."

```